```
import cv2
import mediapipe as mp
import math
import time
import keyboard
from pycaw.pycaw import AudioUtilities,
IAudioEndpointVolume
from comtypes import CLSCTX_ALL

# ------------------- Volume control setup
(Windows) -------------------
devices = AudioUtilities.GetSpeakers()
interface =
devices.Activate(IAudioEndpointVolume.iid,
CLSCTX_ALL, None)
volume =
interface.QueryInterface(IAudioEndpointVolum
e)

def change_volume(delta):
    """Increase or decrease system volume by
delta (0.0 to 1.0)."""
    current =
volume.GetMasterVolumeLevelScalar()
    new_vol = min(1.0, max(0.0, current + delta))

volume.SetMasterVolumeLevelScalar(new_vol,
None)
```

```
# ------------------- MediaPipe setup
------------------
mp_hands = mp.solutions.hands
hands = mp_hands.Hands(
    static_image_mode=False,
    max_num_hands=1,
    min_detection_confidence=0.7,
    min_tracking_confidence=0.5
)
mp_draw = mp.solutions.drawing_utils

# ------------------- Gesture classification
------------------
def get_finger_state(landmarks, tip_idx,
pip_idx, mcp_idx):
    """
    Determine if a finger is extended.
    Uses the angle between the vectors
(MCP→PIP) and (PIP→TIP).
    If angle < 30° → extended, else folded.
    """
    mcp = landmarks[mcp_idx]
    pip = landmarks[pip_idx]
    tip = landmarks[tip_idx]

    # Vectors
    v1 = (pip.x - mcp.x, pip.y - mcp.y, pip.z -
mcp.z)
    v2 = (tip.x - pip.x, tip.y - pip.y, tip.z - pip.z)

    # Normalize and compute angle
```

```
# ------------------ MediaPipe setup ------------------
mp_hands = mp.solutions.hands
hands = mp_hands.Hands(
    static_image_mode=False,
    max_num_hands=1,
    min_detection_confidence=0.7,
    min_tracking_confidence=0.5
)
mp_draw = mp.solutions.drawing_utils

# ------------------ Gesture classification ------------------
def get_finger_state(landmarks, tip_idx, pip_idx, mcp_idx):
    """
    Determine if a finger is extended.
    Uses the angle between the vectors (MCP→PIP) and (PIP→TIP).
    If angle < 30° → extended, else folded.
    """
    mcp = landmarks[mcp_idx]
    pip = landmarks[pip_idx]
    tip = landmarks[tip_idx]

    # Vectors
    v1 = (pip.x - mcp.x, pip.y - mcp.y, pip.z - mcp.z)
    v2 = (tip.x - pip.x, tip.y - pip.y, tip.z - pip.z)

    # Normalize and compute angle
```

```
    # Normalize and compute angle
    norm1 = math.sqrt(v1[0]**2 + v1[1]**2 +
v1[2]**2)
    norm2 = math.sqrt(v2[0]**2 + v2[1]**2 +
v2[2]**2)
    if norm1 == 0 or norm2 == 0:
        return False
    dot = v1[0]*v2[0] + v1[1]*v2[1] + v1[2]*v2[2]
    angle = math.degrees(math.acos(max(-1,
min(1, dot / (norm1 * norm2)))))
    return angle < 30  # extended if almost
straight

def classify_gesture(landmarks):
    """
    Returns: 'THUMBS_UP', 'FIST', 'OPEN_PALM',
or 'UNKNOWN'.
    """
    # Finger tip, pip, mcp indices for each finger
(MediaPipe)
    fingers = {
        'thumb':  (4, 3, 2),
        'index':  (8, 6, 5),
        'middle': (12, 10, 9),
        'ring':   (16, 14, 13),
        'pinky':  (20, 18, 17)
    }
```

```
    extended = {}
    for name, (tip, pip, mcp) in fingers.items():
        extended[name] =
get_finger_state(landmarks, tip, pip, mcp)

    # Thumbs up: thumb extended, others
folded
    if extended['thumb'] and not
any([extended['index'], extended['middle'],
                             extended['ring'],
extended['pinky']]):
        return 'THUMBS_UP'

    # Fist: none extended
    if not any(extended.values()):
        return 'FIST'

    # Open palm: all extended
    if all(extended.values()):
        return 'OPEN_PALM'

    return 'UNKNOWN'

# ------------------ Action handling with
cooldown ------------------
last_action_time = 0
action_cooldown = 1.0  # seconds
```

```
def perform_action(gesture):
    global last_action_time
    current_time = time.time()
    if current_time - last_action_time <
action_cooldown:
        return  # too soon

    if gesture == 'THUMBS_UP':
        print("▶️ Play/Pause")
        keyboard.press_and_release('play/pause')
# media key
        last_action_time = current_time
    elif gesture == 'FIST':
        print("🔉 Volume Down")
        change_volume(-0.05)  # decrease 5%
        last_action_time = current_time
    elif gesture == 'OPEN_PALM':
        print("🔊 Volume Up")
        change_volume(0.05)   # increase 5%
        last_action_time = current_time

# ------------------- Main webcam loop
-------------------
cap = cv2.VideoCapture(0)
if not cap.isOpened():
    print("Error: Could not open webcam.")
    exit()

print("Press 'q' to quit.")
```

```
while True:
    success, frame = cap.read()
    if not success:
        print("Failed to capture frame.")
        break

    # Flip for mirror view and convert to RGB
    frame = cv2.flip(frame, 1)
    rgb = cv2.cvtColor(frame,
cv2.COLOR_BGR2RGB)
    result = hands.process(rgb)

    gesture = 'UNKNOWN'
    if result.multi_hand_landmarks:
        for hand_landmarks in
result.multi_hand_landmarks:
            # Draw landmarks
            mp_draw.draw_landmarks(frame,
hand_landmarks,
mp_hands.HAND_CONNECTIONS)
            # Classify gesture
            gesture =
classify_gesture(hand_landmarks.landmark)
            perform_action(gesture)

    # Show gesture on frame
    cv2.putText(frame, f"Gesture: {gesture}",
(10, 50),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0,
255, 0), 2)
```

```
    result = hands.process(rgb)

    gesture = 'UNKNOWN'
    if result.multi_hand_landmarks:
        for hand_landmarks in
result.multi_hand_landmarks:
            # Draw landmarks
            mp_draw.draw_landmarks(frame,
hand_landmarks,
mp_hands.HAND_CONNECTIONS)
            # Classify gesture
            gesture =
classify_gesture(hand_landmarks.landmark)
            perform_action(gesture)

    # Show gesture on frame
    cv2.putText(frame, f"Gesture: {gesture}",
(10, 50),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0,
255, 0), 2)
    cv2.imshow('Hand Gesture Recognition',
frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```